

Playwright BDD — Interview Q&A Guide

Interview preparation for Senior QE roles, based on this project's framework, decisions, and real CI debugging experience.

Part 1 — Your Project (Answer With Your Repo)

Use these answers as a base. Personalize with your own wording in the interview.

Q1. Walk me through your test automation framework architecture.

Answer:

Our framework has four layers:

1. **Gherkin features** (`features/login.feature`) — business-readable scenarios
2. **Step definitions** (`steps/login.step.ts`) — thin glue that maps Gherkin to code
3. **Page objects** (`pages/LoginPage.ts`) — locators, actions, and assertions
4. **Playwright Test runner** — browser control, parallelism, reporting, CI

`bddgen` converts `.feature` files into native Playwright tests in `.features-gen/`.
Playwright's `webServer` seeds test data and starts the Automation Practice Store before tests run. Custom fixtures in `steps/steps.ts` inject a `LoginPage` instance per test so parallel runs stay isolated.

Q2. Why playwright-bdd instead of plain Playwright or CucumberJS?

Answer:

Plain Playwright is great for developers but scenarios aren't readable for all stakeholders. CucumberJS with Playwright as a library loses Playwright Test runner features — parallel sharding, built-in traces, fixtures, and HTML reports.

playwright-bdd gives us both: Gherkin for readability and the full Playwright Test runner under the hood. We get BDD syntax without sacrificing Playwright's native capabilities.

Q3. Why fixtures instead of `let loginPage` at module level?

Answer:

With `fullyParallel: true`, multiple tests run in the same worker process simultaneously. A module-level variable is shared state — one test can overwrite another's `loginPage` reference, causing race conditions and flaky failures.

Playwright fixtures create a fresh `LoginPage` per test automatically. Each test gets its own instance tied to its own `page`, which is safe for parallel execution. This is the recommended pattern for page objects in Playwright.

Q4. Where do locators live, and why?

Answer:

All locators live in page objects (`LoginPage.ts`), not in step definitions. Steps only call methods like `loginPage.enterUsername()` or `loginPage.verifyLoggedIn()`.

If a selector changes, we update one file. Steps stay stable and readable. This follows the Single Responsibility Principle — steps orchestrate behaviour; pages know the UI.

Q5. What do you automate vs not automate?

Answer:

Automate: - Critical user journeys (login, logout, checkout) - Regression on every PR (via CI) - Negative paths with clear expected outcomes (invalid credentials, required field validation)

Don't automate (or lower priority): - One-off exploratory findings - Pure visual design review (unless using visual regression selectively) - Tests that duplicate unit test coverage (e.g. password regex logic — test at API/unit level) - Third-party integrations we don't control (mock or contract-test instead)

We follow the test pyramid: many unit/API tests, fewer focused E2E tests for critical paths.

Q6. What's still missing for strong login coverage?

Answer:

Current coverage: happy path, logout, invalid credentials, empty field validation.

Gaps I'd add next: - **Remember me** checkbox behaviour and session persistence - **Session expiry** / protected route redirect when not logged in - **Multiple invalid attempts** (rate limiting if applicable) - **Accessibility** — keyboard navigation, ARIA labels on error alerts - **Cross-browser** — Firefox/WebKit in CI, not just Chromium - **API-level login** for faster test setup (test pyramid)

Q7. Why assert `validity.valueMissing` instead of tooltip text?

Answer:

Empty required fields trigger native HTML5 validation. The tooltip ("Please fill in this field.") is browser-rendered UI — not a DOM element. The JavaScript API exposes `validationMessage`, but even that varies by browser ("fill in" vs "fill out").

Asserting exact tooltip text is brittle. `validity.valueMissing === true` confirms the field failed required validation regardless of browser wording. For app-rendered errors (`data-testid="login-error"`), we assert DOM text because the app controls that message.

Q8. Tests passed locally but failed in CI. What happened?

Answer:

Login tests failed because `Logout` was never visible — login never succeeded.

Root cause: user data is stored in gitignored Excel files (`backend/data/*.xlsx`). Locally I had run `npm run seed`, creating `testuser / Test@123`. CI only ran `npm ci` — no seed — so the user database was empty. Login returned "Invalid username or password."

Validation tests still passed because they don't need seeded users.

Fix: Added `npm run seed` to the Playwright `webServer` command so data is seeded before the app starts in both local CI and GitHub Actions.

Q9. How does your GitHub Actions pipeline work?

Answer:

On every push and pull request:

1. Checkout code
2. Install app dependencies (`npm ci` in `automation-practice-store`)
3. Install test dependencies (`yarn install` in `playwrightBDD`)
4. Install Playwright Chromium with system deps
5. Run `yarn test` — which runs `bddgen` , starts `webServer` (seed + app), executes tests in parallel
6. On failure, upload HTML report as a 14-day artifact for debugging

Playwright manages the app lifecycle via `webServer` — we don't manually start/stop the server in the workflow.

Q10. A test is flaky — only fails in CI sometimes. How do you debug?

Answer:

1. **Download CI artifacts** — HTML report, screenshots, video, trace
2. **Enable trace on first retry** (already in config) — step-by-step timeline
3. **Re-run locally with CI conditions** — `CI=true yarn test` , single worker (`--workers=1`) to rule out parallel issues
4. **Check for shared state** — module variables, test data collisions, hardcoded users modified by parallel tests
5. **Replace fixed waits** with Playwright auto-waiting assertions (`toBeVisible` , `toHaveText`)
6. **Check environment** — seed data, timing, network, headless vs headed differences

If still flaky after 3+ investigation cycles, quarantine the test and fix root cause before re-enabling.

Q11. `getByTestId` VS `#id` VS `getByRole` — when to use each?

Answer:

Locator	Targets	When to use
<code>getByRole('button', { name: 'Login' })</code>	Accessible role + name	Preferred for buttons, links, headings — mirrors how users and screen readers interact
<code>getByTestId('login-submit-button')</code>	<code>data-testid</code> attribute	Stable test hooks when role/label isn't unique
<code>locator('#username')</code>	HTML <code>id</code> attribute	Works, but ids may change; not the same as <code>data-testid</code>

Priority: **role** → **test id** → **css**. We use `getByTestId` for form fields and `getByRole` for buttons in this project.

Q12. Why are Gherkin step parameters separate function arguments in playwright-bdd?

Answer:

In Playwright-style playwright-bdd, the first callback argument is **only for fixtures** (`{ page }` , `{ loginPage }`). Gherkin capture groups like `{string}` are passed as **additional arguments** after the fixtures object.

```
````typescript // Correct When('User enter the username as {string} and password as {string}', async ({ loginPage }, username, password) => { ... });
```

```
// Wrong — TypeScript error; username/password are not fixtures When('...', async ({ loginPage, username, password }) => { ... }); ````
```

---

## Q13. What does `And` mean in a feature file if steps use Given/When/Then?

### Answer:

`And` and `But` are Gherkin shorthand. They inherit the keyword from the previous step:

- `And` after `Given` → treated as **Given**
- `And` after `When` → treated as **When**
- `And` after `Then` → treated as **Then**

In step code we always register with `Given` , `When` , or `Then` — never `And` . bddgen resolves `And` user should see the homepage to a `Then` step definition.

---

## Part 2 — Must-Know Playwright Questions (Senior QE)

General Playwright knowledge expected at senior level, beyond this project.

---

## Q14. What is Playwright's auto-waiting and why does it matter?

### Answer:

Playwright automatically waits for elements to be actionable before clicking, filling, or asserting. Actions retry until timeout instead of failing immediately on a missing element.

This reduces flaky tests caused by manual `sleep()` or premature interactions. Prefer Playwright assertions ( `expect(locator).toBeVisible()` ) over `waitForTimeout()` .

---

## Q15. Difference between `page.click()` and `locator.click()` ?

Answer:

`page.click(selector)` is legacy-style — takes a selector string. `locator.click()` uses a Locator object with auto-waiting and strict mode (fails if multiple elements match).

Best practice: create locators with `getByRole` , `getByTestId` , etc., then act on the locator. Locators are reusable and chainable.

---

## Q16. What is strict mode?

Answer:

Playwright locators enforce strict mode by default — an action fails if the locator matches **more than one** element. This catches ambiguous selectors early.

Fix: narrow the locator ( `getByRole('button', { name: 'Login' })` ) or use `.first()` / `.nth(0)` only when duplication is intentional.

---

## Q17. How do Playwright fixtures work?

Answer:

Fixtures set up and tear down test dependencies (browser, page, custom objects). Built-in: `page` , `context` , `browser` . Custom fixtures extend `test` via `test.extend()` .

Fixtures are **lazy** (created when used), **scoped** (test/worker), and **isolated** per test by default — ideal for page objects, API clients, and test data.

```
typescript export const test = base.extend<{ loginPage: LoginPage }>({
 loginPage: async ({ page }, use) => { await use(new LoginPage(page)); },
});
```

---

## Q18. Test scope vs worker scope fixtures?

Answer:

| Scope | Lifecycle | Use for | |-----|-----|-----| | **Test** (default) | Created per test | Page objects, fresh state | | **Worker** | Shared across tests in same worker | Expensive setup (DB connection, auth token cache) — use carefully |

Worker-scoped shared state can cause parallel test pollution. Default to test scope unless there's a clear performance need.

---

## Q19. How do you handle authentication without logging in through UI every test?

### Answer:

Senior approach (test pyramid):

1. **API login** — POST credentials, save token/cookies
2. **Inject storage state** — `context.addCookies()` or `page.goto` with saved `storageState`
3. **Reuse via fixture** — login once in `beforeAll`, save `storageState` to file, reuse in tests

UI login tests validate the login flow. Other tests skip UI login and start authenticated — faster and more stable.

```
typescript // playwright.config.ts use: { storageState:
'playwright/.auth/user.json' }
```

---

## Q20. What is `storageState` and when do you use it?

### Answer:

`storageState` captures cookies and localStorage from a browser context. Save after login, load in subsequent tests to skip repetitive auth steps.

Use for: speeding up suites, testing post-login flows. Don't use as a substitute for at least one dedicated login E2E test.

---

## Q21. How do you run tests in parallel safely?

### Answer:

- Enable `fullyParallel: true` for independent tests

- Avoid shared mutable state (module variables, shared test users modified by tests)
  - Use unique test data per test (UUID suffix on usernames)
  - Isolate with separate browser contexts per test (Playwright default)
  - Use fixtures for page objects — one instance per test
  - For shared backend state, use API cleanup in `afterEach` or dedicated test tenants
- 

## Q22. What is trace and when do you enable it?

### Answer:

Trace records a full timeline: actions, DOM snapshots, network, console, screenshots. View with `npx playwright show-trace trace.zip`.

Config options: - `on-first-retry` — capture on flaky retry (good default for CI) - `on` — every test (heavy, use for debugging sessions) - `retain-on-failure` — keep only failed test traces

Essential for debugging CI-only failures.

---

## Q23. How do you handle file uploads and downloads?

### Answer:

**Upload:** `typescript await page.getByTestId('file-input').setInputFiles('path/to/file.pdf');`

**Download:** `typescript const downloadPromise = page.waitForEvent('download'); await page.getByRole('link', { name: 'Export' }).click(); const download = await downloadPromise; await download.saveAs('report.csv');`

---

## Q24. How do you mock API responses in Playwright?

### Answer:

Use `page.route()` to intercept network requests:

```
typescript await page.route('**/api/products', (route) => route.fulfill({
status: 200, body: JSON.stringify({ products: [] }), }));
```

Use for: isolating UI from backend instability, testing error states, speeding up tests. Don't over-mock — keep some integration tests against real API.



---

## Q25. Difference between `soft` assertions and hard assertions?

### Answer:

Hard assertion ( `expect` ) stops the test immediately on failure.

Soft assertion ( `expect.soft` ) records failure but continues the test — useful for checking multiple things on one page before teardown.

```
typescript await expect.soft(page.getByTestId('title')).toBeVisible();
await expect.soft(page.getByTestId('price')).toHaveText('$10'); // Test
fails at end if any soft assertion failed
```

Use soft assertions sparingly — hard failures are easier to debug.

---

## Q26. How do you test across multiple browsers?

### Answer:

Define projects in `playwright.config.ts` :

```
typescript projects: [{ name: 'chromium', use: { ...devices['Desktop
Chrome'] } }, { name: 'firefox', use: { ...devices['Desktop Firefox'] } }, {
name: 'webkit', use: { ...devices['Desktop Safari'] } },],
```

Run: `npx playwright test --project=firefox`

Senior decision: run full matrix on nightly; smoke on Chromium only for PR speed.

---

## Q27. What is the Page Object Model and what are its limits?

### Answer:

POM encapsulates page structure and behaviour in a class. Steps/tests call high-level methods, not raw selectors.

**Benefits:** maintainability, reuse, readable tests.

**Limits:** POM can become a "god class" if every page method lands in one file. Split by page/component. Don't put assertions in tests AND pages inconsistently — pick a convention (we assert in page objects).

Alternative: **Screenplay pattern** or **Component objects** for large SPAs.

---

## Q28. How do you reduce test execution time?

### Answer:

1. API setup instead of UI login
  2. `storageState` for auth reuse
  3. Parallel execution + CI sharding ( `--shard=1/4` )
  4. Run smoke on PR, full suite nightly
  5. Mock slow third-party calls
  6. Block unnecessary resources (images, analytics) via routing
  7. Headless mode in CI
- 

## Q29. What are common anti-patterns in Playwright automation?

### Answer:

| Anti-pattern | Better approach | |-----|-----| |  
| `page.waitForTimeout(5000)` | Auto-waiting assertions | | Selectors like `div > div > span:nth(3)` | `getByRole` , `getByTestId` | | Shared global state between tests |  
Fixtures, isolated data | | Testing everything through UI | API tests for business logic | |  
Copy-paste scenarios | Scenario Outline / shared steps | | Ignoring CI failures | Fix or quarantine with ticket | |  
Hardcoded prod URLs/credentials | Config, env vars, secrets |

---

## Q30. How do you integrate Playwright into CI/CD effectively?

### Answer:

1. Install browsers with `--with-deps` on Linux CI
  2. Start app via `webServer` or docker-compose
  3. Seed test data as part of environment setup
  4. Upload artifacts on failure (report, trace, video)
  5. Fail fast on PR; optional non-blocking nightly for full matrix
  6. Cache dependencies ( `actions/cache` , yarn/npm cache)
  7. Set `retries: 2` in CI only for known infra flakiness — don't mask real bugs
- 

## Q31. BDD: when is it a good idea vs overkill?

### Answer:

**Good fit:** - QA, product, and dev need shared readable specs - Regulatory/compliance requires human-readable test cases - Behaviour-driven collaboration is part of team culture

**Overkill:** - Small dev-only team with strong TypeScript skills - Simple CRUD with no stakeholder review of scenarios - BDD files become duplicate documentation nobody maintains

Rule: BDD adds value when scenarios are **read and reviewed** by non-coders. Otherwise plain Playwright may be simpler.

---

## Q32. How do you report test results to stakeholders?

**Answer:**

- **Playwright HTML report** — developers and QA (traces, screenshots)
- **CI status checks** — block merge on failure
- **JUnit/XML export** — integrate with Jenkins, Azure DevOps
- **Allure** — richer dashboards for management
- **Summary metrics** — pass rate, duration trend, flaky test count

Senior QE translates technical results into risk: "Login regression blocked release" vs "47 tests passed."

---

## Part 3 — Quick Reference Cheat Sheet

### Locator priority

```
getByRole → getByLabel → getByPlaceholder → getByTestId → CSS → XPath (last resort)
```

### playwright-bdd step signature

```
typescript When('step with {string}', async ({ fixture }, param) => { ... });
```

### Native vs app validation

```
````typescript // App error (DOM) await expect(page.getByTestId('login-error')).toHaveText('Invalid username or password');
```

```
// Browser validation (no DOM element) await expect(page.getByTestId('password-  
input')).toHaveJSProperty('validity.valueMissing', true); ``
```

CI environment parity

```
Local: npm run seed → users exist CI: must seed before tests → same state
```

Part 4 — Practice Plan

| Day | Activity | |-----|-----| | 1 | Answer Q1–Q7 out loud (framework + strategy) | | 2 | Answer Q8–Q13 out loud (CI + Playwright specifics) | | 3 | Answer Q14–Q22 (senior Playwright concepts) | | 4 | Answer Q23–Q32 (advanced topics) | | 5 | Mock interview: explain repo end-to-end in 5 minutes without notes |

Built from the CollectiveFrameworks playwrightBDD project — login BDD suite with CI on GitHub Actions.